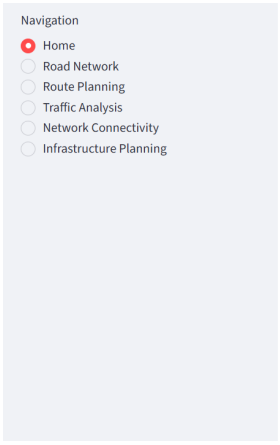
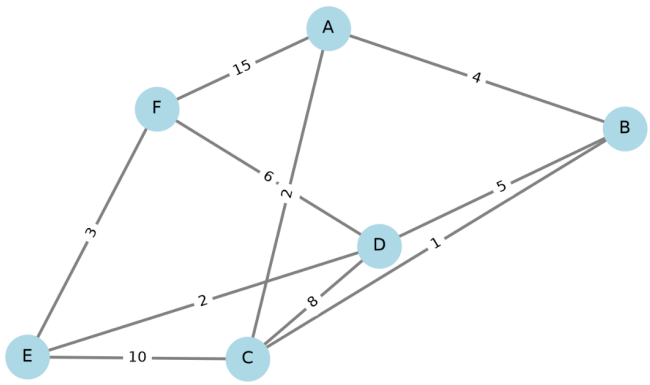
K.R. MANGALAM UNIVERSITY

# SMART CITY TRAFFIC MANAGEMENT SYSTEM

DAA CAPSTONE PROJECT — FINAL REPORT

| | |
|---|---|
| **Student Name** | Harshit Kumar |
| **Roll Number** | 2401201057 |
| **Programme** | BCA (AI & Data Science) |
| **Semester** | V |
| **Course Code** | ENCA351 |
| **Course Name** | Design and Analysis of Algorithms Lab |
| **Faculty** | Dr. Aarti |
| **Academic Session** | 2026–27 |




Route Planning • Traffic Analysis • Connectivity • Infrastructure Optimization

# CONTENTS

1. Abstract

2. Introduction and Problem Statement

3. Objectives and Scope

4. Technologies Used

5. System Design and Modules

6. Algorithms Implemented

7. Application Screenshots and Outputs

8. Results and Testing

9. Performance Analysis

10. Limitations and Future Enhancements

11. Conclusion

12. References

# 1. ABSTRACT

The Smart City Traffic Management System is a Python-based prototype that demonstrates how graph algorithms can support road-network planning. Intersections are represented as vertices and roads as weighted edges, where each weight represents a generalized travel or infrastructure cost. The system integrates Dijkstra's algorithm for shortest-route planning, Bellman–Ford for route recalculation after simulated congestion, Breadth-First Search (BFS) and Depth-First Search (DFS) for traversal and connectivity, and Prim's and Kruskal's algorithms for minimum spanning tree (MST) planning. A Streamlit interface provides interactive pages for road modeling, route planning, traffic analysis, network connectivity, infrastructure planning and performance comparison. A Jupyter notebook contains algorithm demonstrations and experimental outputs.

# 2. INTRODUCTION AND PROBLEM STATEMENT

Urban road networks contain intersections connected by roads with different travel costs. As a network grows, manually choosing a route becomes difficult. Congestion or a road incident can also change which route is the most efficient. City planners need tools to identify low-cost routes, determine whether all intersections remain reachable, and plan a connected network without unnecessary infrastructure cost.

Graph theory provides a useful way to represent these problems. Intersections are modeled as vertices, roads as edges, and road weights as travel or infrastructure costs. The algorithms implemented in this project solve different tasks on the same graph, allowing their behavior to be inspected through an interactive application.

## Problem Statement

To develop an interactive prototype that models a city road network and demonstrates shortest-route planning, simulated congestion analysis, network connectivity, minimum spanning tree planning and execution-time comparison across different graph sizes.

# 3. OBJECTIVES AND SCOPE

The main objectives of the project are:
1. Represent intersections and roads using a weighted graph.
2. Calculate a minimum-cost route between a selected source and destination.
3. Simulate congestion by increasing a road's cost and recalculating the route.
4. Use BFS and DFS to inspect traversal order and reachability.
5. Calculate a minimum spanning tree using Prim's and Kruskal's algorithms.
6. Compare execution time for six algorithms on graph sizes of 10, 25, 50 and 100 intersections.
7. Present results through a Streamlit interface and export benchmark data as CSV.

## Project Scope

This project is an educational prototype based on an illustrative road network with manually assigned weights. It does not currently use live traffic feeds, GPS tracking or real geographic coordinates. Congestion is simulated by changing the weight of a selected road.

# 4. TECHNOLOGIES USED

| Technology | Purpose |
|---|---|
| Python | Core application logic and algorithm integration. |
| Streamlit | Interactive browser interface, navigation, forms and result display. |
| NetworkX | Graph structures, shortest paths, traversal, connectivity and MST operations. |
| Matplotlib | Road-network visualization and route highlighting. |
| Pandas | Benchmark result tables and CSV export. |
| Jupyter Notebook | Step-by-step algorithm demonstrations and output analysis. |
| Git / GitHub | Version control and project sharing, if configured. |

# 5. SYSTEM DESIGN AND MODULES

The system is organized into separate files so that the user interface and algorithm logic remain manageable. The Streamlit application initializes a default undirected weighted graph and provides navigation for each major function. The relevant helper function is called when a user selects an operation.

| File | Responsibility |
|---|---|
| app.py | Streamlit navigation, input controls, graph rendering, result tables and downloads. |
| route_planner.py | Calculates a minimum-cost route using Dijkstra's algorithm. |
| traffic_analysis.py | Compares route and cost before and after a road-cost increase. |
| connectivity_module.py | Connectivity check, BFS/DFS traversal and connected components. |
| mst_planner.py | Minimum spanning tree using Prim or Kruskal. |
| benchmark.py | Measures algorithm execution time on synthetic graphs. |
| project_notebook.ipynb | Algorithm demonstrations, printed results and performance plots. |
| requirements.txt | Lists Python packages required to run the project. |
| README.md | Provides project overview and setup instructions. |
| Screenshots/ | Contains screenshots captured during application testing. |

### Application Workflow

1. Load the default city graph or add intersections and roads.
2. Choose a module from the sidebar.
3. Select the required source, destination, road or algorithm.
4. Execute the operation and inspect the displayed output.
5. View the graph or export benchmark results when required.

# 6. ALGORITHMS IMPLEMENTED

| Algorithm | Purpose | Typical complexity |
|---|---|---|
| Dijkstra | Finds a minimum-cost route with non-negative edge weights. | O((V + E) log V) with priority queue |
| Bellman–Ford | Recalculates shortest paths; supports negative edge weights when no negative cycle invalidates the result. | O(VE) |
| BFS | Visits vertices breadth-first; useful for reachability. | O(V + E) |
| DFS | Explores a branch deeply before backtracking. | O(V + E) |
| Prim | Builds an MST by adding low-cost connecting edges. | O(E log V) with priority queue |
| Kruskal | Adds edges in increasing weight order while avoiding cycles. | O(E log E) |

### 6.1 Dijkstra — Route Planning

The user selects a source and destination. The application calculates a minimum-cost route and displays the route sequence and total cost. The selected route is highlighted on the graph. Dijkstra's algorithm is suitable for this feature because the road costs are non-negative.

### 6.2 Bellman–Ford — Traffic Analysis

A selected road receives an additional congestion cost on a copy of the graph. The system computes the route before and after the cost change while preserving the original graph for comparison. In the notebook demonstration, the original route cost was 13; after the selected road's cost increased, the route changed to A → C → E → F with a cost of 15.

## 6.3 BFS, DFS AND NETWORK CONNECTIVITY

Breadth-First Search (BFS) visits neighboring vertices in layers, while Depth-First Search (DFS) explores one branch before backtracking. Both algorithms return a traversal order from the selected starting intersection. The connectivity function checks whether all vertices belong to a single connected component. In the notebook demonstration, the graph was connected and contained six intersections.

## 6.4 PRIM AND KRUSKAL — INFRASTRUCTURE PLANNING

A minimum spanning tree connects all vertices in a connected, undirected graph without cycles and with the lowest possible total edge weight. Prim's algorithm grows a tree by adding a low-cost edge that connects a new vertex. Kruskal's algorithm considers edges in ascending order and skips edges that would form a cycle. In the notebook test, both algorithms returned five edges and a total cost of 13.

# 7. APPLICATION SCREENSHOTS AND OUTPUTS

### Home Dashboard

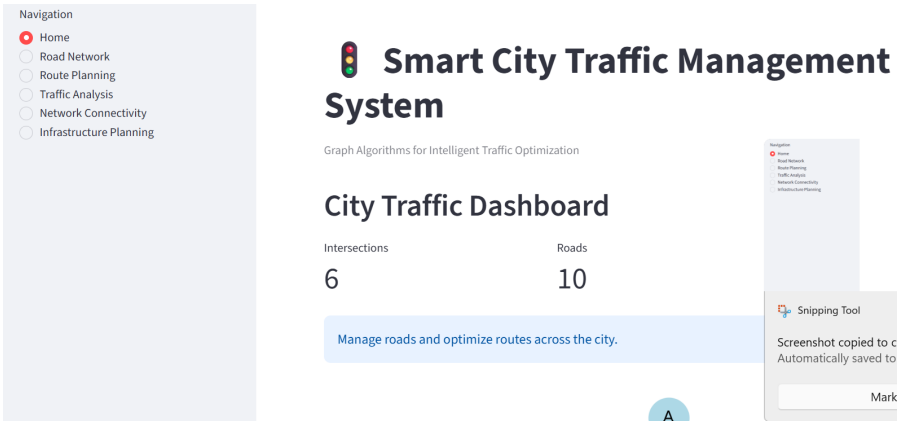



Figure 1. Streamlit dashboard showing the city overview and navigation.

### Road Network and Route Visualization

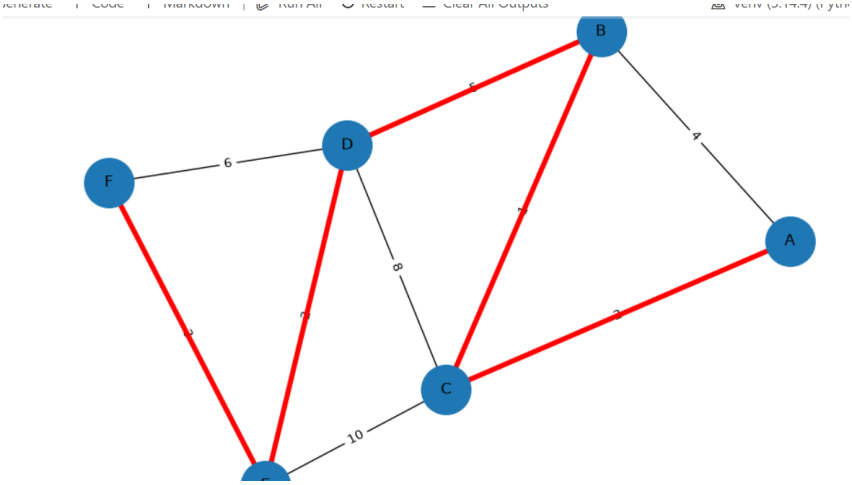



Figure 2. Road network with selected route edges highlighted.

## Performance Benchmark Results

| | Intersections | Roads | Algorithm | Execution Time (ms) |
|---|---|---|---|---|
| 0 | 10 | 30 | BFS | 1.7194 |
| 1 | 10 | 30 | DFS | 0.7003 |
| 2 | 10 | 30 | Dijkstra | 0.6993 |
| 3 | 10 | 30 | Bellman-Ford | 0.437 |
| 4 | 10 | 30 | Prim | 1.2679 |
| 5 | 10 | 30 | Kruskal | 0.6996 |
| 6 | 25 | 75 | BFS | 0.1757 |
| 7 | 25 | 75 | DFS | 0.1488 |
| 8 | 25 | 75 | Dijkstra | 0.0985 |
| 9 | 25 | 75 | Bellman-Ford | 0.2154 |

Algorithm Execution Time

Figure 3. Benchmark table showing execution-time measurements.

## Algorithm Execution-Time Chart

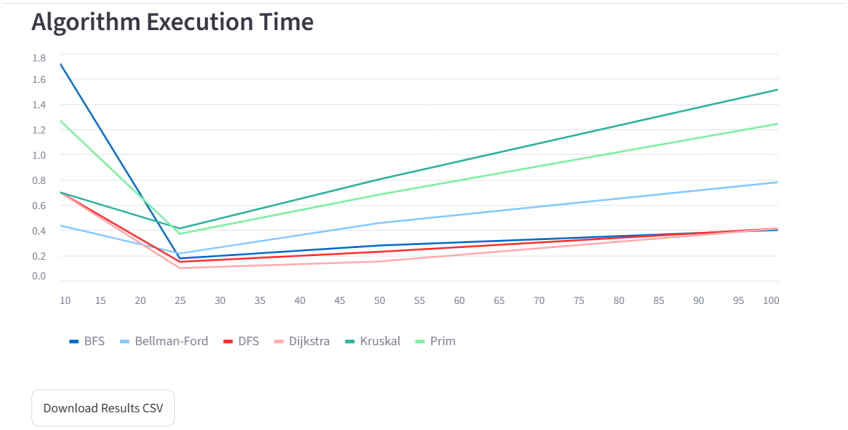



Figure 4. Execution-time comparison across graph sizes.

## Network Connectivity Output

The connectivity module was tested after the missing connected-components function was corrected. The successful output showed that the network was connected, with six intersections and one connected component. BFS and DFS controls were available to display traversal order from a selected start. The earlier error screenshot has been omitted.

# 8. RESULTS AND TESTING

| Test | Observed result | Interpretation |
|---|---|---|
| Notebook road network | 6 intersections and 9 roads. | Graph created and visualized. |
| Dijkstra route | A → C → B → D → E → F; total cost 13. | Shortest-route demonstration completed. |
| Traffic simulation | Updated route A → C → E → F; cost 15 after a road-cost increase. | Route recalculated after simulated congestion. |
| BFS / DFS | Traversal outputs produced; network connected. | All notebook graph vertices were reachable. |
| Prim / Kruskal | Both MST total costs were 13; five edges. | Matching MST costs confirmed. |
| Performance page | Results table, line chart and CSV download appeared. | Benchmark interface tested. |

# 9. PERFORMANCE ANALYSIS

The benchmark compares BFS, DFS, Dijkstra, Bellman–Ford, Prim and Kruskal on synthetic graphs containing 10, 25, 50 and 100 intersections. Execution time is measured using Python's high-resolution performance counter and reported in milliseconds. The Streamlit page displays a table and line chart and provides a CSV download option.

Timing values can vary between runs because of system load, interpreter overhead and the short duration of operations on small graphs. A single run should therefore be treated as a local experiment rather than a universal ranking. A more rigorous experiment would repeat each algorithm several times on the same graph instances and compare median or average timings.

## Theoretical interpretation

BFS and DFS have linear traversal complexity in the number of vertices and edges. Dijkstra is designed for shortest paths with non-negative weights. Bellman–Ford performs more edge-relaxation work but supports negative weights. Prim and Kruskal solve the same minimum spanning tree problem using different strategies. Their matching total cost in the notebook provides a consistency check for that graph.

## Graph Version Note

**The notebook demonstration used six intersections and nine roads. The Streamlit application's default graph includes an additional A–F road, so it starts with six intersections and ten roads. Some route and cost outputs may differ between the notebook and the application.**

# 10. LIMITATIONS AND FUTURE ENHANCEMENTS

## Current Limitations

1. The network is illustrative and does not use real geographic coordinates.
2. Edge weights represent generalized costs rather than calibrated travel times or distances.
3. Congestion is simulated manually instead of being obtained from a live traffic service.
4. Benchmark graphs are synthetic, and execution times depend on the local environment.
5. The prototype does not include live traffic APIs, persistent database storage or user authentication.

## Future Enhancements

1. Integrate real road-network data and map-based visualization.
2. Connect to live traffic services to update road costs dynamically.
3. Store network data, incidents and benchmark runs in a database.
4. Repeat benchmark trials and report averages, medians and standard deviations.
5. Add automated tests for disconnected graphs and invalid inputs.
6. Extend the interface to handle larger networks and multiple user roles.

# 11. CONCLUSION

The Smart City Traffic Management System demonstrates how graph algorithms can support route planning, simulated congestion analysis, connectivity inspection and infrastructure planning. The Streamlit interface brings these tasks together in one interactive prototype, while the Jupyter notebook records algorithm demonstrations and performance experiments. Testing showed that routes can be recalculated after a road-cost change, the connectivity module reports the network as connected, and Prim's and Kruskal's algorithms produce the same minimum spanning tree cost for the notebook graph. Although the prototype does not use live traffic information, it provides a foundation for future work using real road data and dynamic costs.

# 12. REFERENCES


1. NetworkX Documentation — https://networkx.org/documentation/stable/
2. Streamlit Documentation — https://docs.streamlit.io/
3. Python Time Module — https://docs.python.org/3/library/time.html
4. Matplotlib Documentation — https://matplotlib.org/stable/
5. Pandas Documentation — https://pandas.pydata.org/docs/